

ARIN 5305 (Fall 2025) Programming Assignment

Prompting, Comparing, Programming, and Debugging

Instructor: Prof. Shing-Chi Cheung

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology

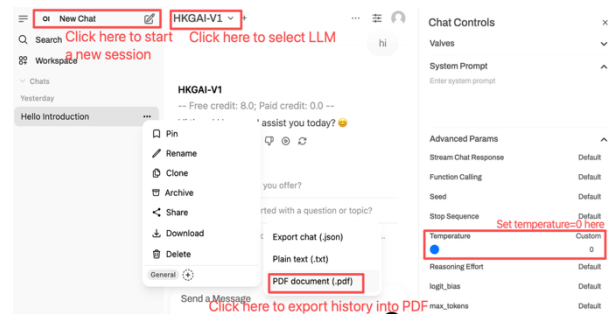
Submission Due Dates: Stage 1 (23:50, Sep 28, 2025); Stage 2 (23:50, Oct 18, 2025); Stage 3 (23:50, Oct 25, 2025)
(Contact TA via Canvas messages or i9s.chen@connect.ust.hk if you encounter difficulties in understanding the tasks)

1. General Information

Assignment Objective: This assignment aims to consolidate the learning of using AI techniques (e.g., LLMs) together with popular software development utilities (e.g., Git/GitHub, test generation tools, and code similarity metrics) to facilitate common software development activities, including code summarization, test preparation, bug localization and repair, and code translation. Through these tasks, students will gain hands-on experience in applying AI techniques to software engineering and a deeper understanding of both the potential and the limitations of these techniques.

Generative AI Usage Policy: In this assignment, we practice using Generative AI tools (LLMs) for software development activities. Please follow the instructions below to use LLMs.

- Use **hkgai-v1** model and set **temperature=0.0** on HKUST GenAI OpenWebUI platform (<https://openwebui.hkust.edu.hk/>). Failing to use the specified LLM and configuration on the platform will lead to a penalty of 10 points at each stage.
- Start a **new chat session** for different LeetCode problems in each assignment task. Failing to do so will get no score.
- Export all the messages in a chat session into a PDF file and a JSON file using the platform's Download-PDF/JSON function. The PDF and JSON files should include complete chat history for corresponding task. You are **NOT** allowed to tamper with LLM outputs and exported PDF/JSON. The requirements for the content to export for each task are explained in the task requirements below.
- HKUST ITSO's usage help for the platform: <https://itso.hkust.edu.hk/services/general-it-services/generative-ai-tools/openwebui-help>



GitHub Repository Setup: GitHub is a widely used platform for Git version control in software development. You will practice storing your code and documentation in a GitHub repository. Please follow the instructions below.

- Your results of all tasks in this assignment should be maintained within a personal **private** GitHub repository.
- Grant TA's GitHub account **arin5305-25fallta** access to your personal repository (by the deadline of Stage 1).
- Submit an archive .zip of the whole repository to Canvas for each stage. Note that we will grade based on the archive version you upload to Canvas. We will also verify that you regularly push modifications to your GitHub repository in a timely manner; failing to do so will result in a **5-point penalty** deduction at each stage.

Submission Policy: You should follow these submission requirements:

- **Late Submission:** No late work accepted without pre-approval.
- **Plagiarism:** Assignments are strictly individual work. Misuse of generative AI, code plagiarism, or altering logs constitutes a violation of academic integrity, resulting in severe consequences.

Resource & Example: We provide the resource (e.g., programs to summarize/translate) and show the expected file/folder structure of your repository in our GitHub repository (<https://github.com/CastleLab/ARIN5305-25F>).

2. Assignment Description

Stage 1 – Task 0: First GitHub Commit (5 pts)

Objective: Familiarize yourself with Git version control, proper commit metadata, and GitHub repo management.

Task Requirements:

- Create a **private** repository on GitHub and clone it to your local machine.
- Add a file named "README.md" claiming your full name, ITSC account (email address), and your student ID.
- Adjust Git configuration and make your first commit to meet 3 requirements:
Commit author name = ITSC account name.
Commit email = ITSC email address.
Commit message = "Init."
- Grant TA account repository access via "Settings" => "Collaborators" => "Add people". Add TA account "*arin5305-25fallta*".

Submission Content: Submit the link to your private repository via Canvas.

Grading Scheme: 5 points in total, 1pt deduction for each requirement violation.

Stage 1 – Task 1: Code Semantic Representation

> Task 1.1: Code Summarization (15 pts)

Objective: Natural language code summaries facilitate an understanding of programs without requiring knowledge of implementation details, serving as an efficient means of communication during development. This task involves writing concise and accurate code summaries with the help of LLMs, enabling both humans and LLMs to easily understand the code semantics without resorting to the implementation details. In this task, you are required to write semantically accurate summaries such that the advanced LLM, *hkgai-v1*, can reproduce the code based on your summaries. The conciseness is measured by the number of tokens in the code summaries.

Task Requirements:

- Write a concise, accurate code summary with LLM's assistance for the correct Java solution of each of the five LeetCode problems (programs available in "resource/correctJavaPrograms/{problem_id}/Solution.java" in our repository). You are allowed to prompt *hkgai-v1* to further edit the initial summary generated by it. You are also allowed to manually edit the summary. Ensure the final summary is both accurate and concise.
- Validate the accuracy of the summary you wrote. Specifically, create a **new chat**, and then prompt *hkgai-v1* with the template located in "resource/prompts/codegeneration/{problem_id}.md" in our repository, which should be filled with your summary. Extract the Java code generated by *hkgai-v1*, and submit it to LeetCode via the link https://leetcode.com/problems/{problem_id}. Set the target programming language to Java.
- Iteratively revise your summaries with the help of *hkgai-v1* and validate their accuracy. For each problem, record the finalized code summaries, chat history to obtain the summaries, and link to your submissions.
- Five LeetCode problems are used in this assignment. Problem IDs: {"basic-calculator", "count-the-number-of-special-characters-ii", "minimum-cost-good-caption", "robot-collisions", "sum-of-largest-prime-substrings"}.

Submission Content for This Task:

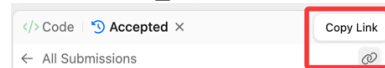
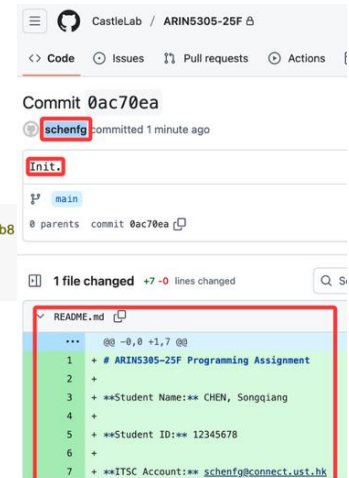
- For each of the five LeetCode problems, record the final summary in "*Task-1/{problem_id}/summary.txt*".
- For each of the five LeetCode problems, record the *LeetCode submission link* of the Java code generated by *hkgai-v1* based on your code summary, in "*Task-1/{problem_id}/submission.txt*".
Link format: https://leetcode.com/problems/{problem_id}/submissions/{submission_id}.

Note: You can copy the link to your submission by clicking this button:

- For each LeetCode problem, dump your chat history of **code generation based on your summary** for your summary from OpenWebUI into a PDF and a JSON file, store them in "*Task-1/{problem_id}/chat.{pdf|json}*".

```
~/Dow/ARIN5305-25F [main +1] git log
commit 0ac70ead1038bd939beffd1c1c266a28015eb1b8
Author: schenfg <schcnfg@connect.ust.hk>
Date:   Sun Aug 31 14:48:21 2025 +0800

    Init.
```



Grading Scheme:

- 5 pts: Correctness, 1 pt per problem if the code regenerated from your summary passes on LeetCode. Summaries must be human-readable descriptive text. Artificial tricks (e.g., encoded strings) will get 0 points.
- 10 pts: Conciseness of code summaries (*measured based on length in tokens split by GPT-4o's tokenizer. Note that we still use hkgai-v1 for experiments. We measure length based on tokens tokenized by GPT 4o's tokenizer.*):
 - ✦ "minimum-cost-good-caption": ≤ 100 tokens: 2 pts; 101–150 tokens: 1.5 pts; 151–200 tokens: 0.5 pts.
 - ✦ For the other four problems: ≤ 30 tokens: 2 pts; 31–50 tokens: 1.5 pts; 51–80 tokens: 0.5 pts.
 - ✦ Websites for counting tokens in given text: <https://platform.openai.com/tokenizer>; <https://www.quizrise.com/token-counter>.

> Task 1.2: Code Similarity Measure (15 pts)

Objective: Measuring code similarity is useful for many activities, e.g., evaluating whether two code snippets are functionally alike, plagiarism detection, clone analysis, and assessing LLM-generated code quality. This task practices the use of automatic similarity metrics to compare code similarity and to analyze whether these metrics align with true semantic equivalence. In this assignment, we practice the CodeBLEU metric.

Task Requirements:

- Implement CodeBLEU calculation. You are allowed to use other libraries and seek assistance from the LLM.
- For each problem, compute CodeBLEU similarity between:
 - ✦ Correct code (in resource/correctJavaPrograms) **vs.** LLM-Reproduced code (submitted to LeetCode in Task1.1)
 - ✦ Correct code (in resource/correctJavaPrograms) **vs.** Faulty code (**in resource/faultyJavaProgramsToCompare**)
- Analyze and discuss whether the metric accurately aligns with semantic consistency.

Submission Content of This Task:

- Put your implementation and scripts for CodeBLEU calculation in "Task-1/codebleu/" folder. Put a README file in the folder to introduce the usage of your implementation, as well as the necessary libraries to install.
- Write a report to discuss the CodeBLEU similarity between the two pairs of code, among all five problems. Include appropriate tables/figures to support your discussion on whether CodeBLEU effectively reflects the semantic consistency. The report should include 400-600 words and saved in "Task-1/codebleu/report.pdf".

Grading Scheme:

- 5 pts: Availability and correctness of CodeBLEU calculation tool and usage instruction.
- 10 pts: Report quality (clarity, correctness, appropriate use of examples/visuals, length, etc.).

*The deadline of Stage 1 is 23:50, Sep 28. Please submit the link to your private repository + the latest zip archive of your repository to **Canvas** by the deadline. Please commit progress to your repository regularly **every week**.*

Stage 2 – Task 2: Test Preparation

> Task 2.1: Automated Random Test Generation (10 pts)

Objective: Automated random testing tools can automatically generate diverse test cases, helping to validate the behaviors of human-written or LLM-generated programs. This task practices the use of Randoop to generate test suites that reach a required coverage. Instructions are available in the appendix.

Task Requirements:

- Use Randoop to generate one test suite called "RegressionTest" for each of the five correct Java programs.
- Each suite must contain **NO more than 10 tests** and achieve **$\geq 40\%$ branch coverage**.
- Measure the branch coverage of each test suite using Eclipse (EclEmma). Other IDEs are NOT allowed.
- Appendix shows basic usage of Randoop. Please adjust parameters to meet the above requirements.

Submission Content of This Task:

- For the correct Java program of each problem, store the random test suite files "RegressionTest.java" and "RegressionTest0.java" generated by Randoop in folder "Task-2/{problem_id}/". Record the command you adopt to generate the test suite in a comment at the beginning of RegressionTest.java (*-0.5pt if it's missed*).

- For each test suite, take a screenshot showing the branch coverage of the test suite on the Solution class (as shown in Fig. 2 in the appendix). Put the screenshot in "[Task-2/{problem_id}/randoop-branch.png/jpg](#)".

Grading Scheme:

- 2 pts per test suite (for the Java solution of each problem) if branch coverage $\geq 40\%$ and with ≤ 10 tests.
- Incorrect screenshots (e.g., not measuring branch coverage, not showing coverage for Solution): -1 pt each.

> Task 2.2: High-Coverage Test Design (15 pts)

Objective: By using Randoop, you will learn that high coverage requires carefully crafted test cases. This task involves designing minimal yet effective test suites that achieve strong branch coverage with only a few test inputs. You can leverage LLM in this task and thereby understand the pros and cons of various techniques.

Task Requirements:

- For the correct Java program of each problem, write a test suite named with *HighCoverageTest*.
- The test suite should include **no more than five test inputs** to the functions in the Solution class and try to achieve the maximum branch coverage. If there are more than five test inputs, we'll grade only the first five.
- The test suite should include appropriate test assertions.
- You are NOT allowed to copy the sample tests in LeetCode problem description. Violation will result in Opt.

Submission Content of This Task:

- Put down the required test suite in "[Task-2/{problem_id}/HighCoverageTest.java](#)" for the correct Java program of each LeetCode problem.
- For each test suite, take a screenshot showing the branch coverage of the test suite on the Solution class (as shown in Fig. 2 in the appendix). Put the screenshot in "[Task-2/{problem_id}/highcoverage-branch.png/jpg](#)".

Grading Scheme: $\sum_{each\ problem} \frac{branch\ coverage\ achieved\ by\ your\ test\ suite}{100\%} \times 3pts$

Stage 2 – Task 3: Bug Localization & Repair (20 pts)

Objective: Debugging of programs is a central activity in software engineering, and LLMs can assist in localizing and fixing faults in programs. This task involves identifying faulty lines, explaining their causes, and proposing fixes with the support of an LLM, while reflecting on the strengths and limitations of AI in debugging.

Task Requirements:

- For the faulty solution of each LeetCode problem in "[resource/faultyJavaProgramsToFix/{problem_id}/Solution.java](#)" in our repository, identify the faulty line. Each faulty program contains a single one-line bug.
- For the faulty solution of each LeetCode problem, explain the symptom caused by the bug.
- For the faulty solution of each LeetCode problem, propose a fix to the identified bug. The patch should be accomplishable in one line of code (including line addition, removal, and content update).

Submission Content of This Task:

- For the faulty Java program of each LeetCode problem, put down the brief description of the identified bug, the buggy Java code line, the suggested patch, and the complete fixed version in "[Task-3/{problem_id}.txt](#)".
- Write a report (300-400 words, with appropriate illustrating examples [words not counted]) to discuss: 1) How LLMs assist you in locating and fixing bugs. 2) Limitations or failure cases of LLM that you observed. Put the report in the path "[Task-3/report.pdf](#)".

Grading Scheme:

- 15 pts: Correct Localization and Repair of Bug. 1.5 pts * 5 problems * 2 steps: identify + repair.
- 5 pts: Report quality (clarity, correctness, appropriate use of examples/visuals, length, etc.).

*The deadline for Stage 2 is 23:50, Oct 18. Please submit the latest zip archive of your repository (incl. all content required by the tasks in Stage 2) to **Canvas** by the deadline. Please commit your progress regularly **every week**.*

Stage 3 – Task 4: Code Translation (20 pts)

Objective: Code translation across programming languages is useful for various development activities, e.g., software migration and interoperability. This task guides you to explore LLM’s capability to translate Python code into Rust, analyze translation correctness, and reflect on the challenges of cross-language code generation.

Task Requirements:

- Translate the correct **Python** solutions of the five problems into **Rust**. The Python solutions to translate are stored in "resource/correctPythonPrograms/{problem_id}.py" in our repository (*no longer Java programs*). Similar to the requirement in Task 1, you are allowed to prompt *hkgai-v1* further to repair the translation initially generated by it. A few minor manual edits are also allowed, but don’t start from scratch manually.
- Validate the correctness of the translation results (Rust programs). Specifically, submit the translation result to LeetCode via the link https://leetcode.com/problems/{problem_id}. Set programming language to Rust.
- Iteratively revise your translation with the LLM’s assistance and validate its correctness. For each problem, record the finalized Rust program, chat history to obtain the translation, and link to your Rust submissions.

Submission Content of This Task:

- For each of the five LeetCode problems, record the final translation result in "Task-4/{problem_id}/main.rs".
- For each of the five LeetCode problems, record the LeetCode submission link of the Rust program translated by you with the help of *hkgai-v1* in "Task-4/{problem_id}/submission.txt".
Link format: https://leetcode.com/problems/{problem_id}/submissions/{submission_id}.
- For each of the five LeetCode problems, **start a new session** to chat, and **export your code translation chat history** into a PDF file and a JSON file. Put the PDF/JSON file in "Task-4/{problem_id}/chat.{pdf|json}".
- Write a report (400-600 words, with appropriate illustrating examples [words not counted]) to discuss:
1) Problems encountered in translation. 2) Suggestions for improving Python-to-Rust translation with LLMs.
Put the report in the path "Task-4/report.pdf".

Grading Scheme:

- 10 pts: Correct Rust translations (2 pts each problem * 5).
- 10 pts: Report quality (clarity, correctness, appropriate use of examples/visuals, length, etc.).

*The deadline of Stage 3 is 23:50, Oct 25. Please submit the latest zip archive of your repository (incl. all content required by the tasks in this stage) to **Canvas** by the deadline. Please regularly commit your progress **every week**.*

Important Notes:

1. Please export both a PDF file and a JSON file from OpenWebUI platform to avoid display issue in PDF.
2. You cannot use LLMs to prepare the content of reports. Violation of this will be considered plagiarism.
3. Please make sure that you submit the right version of your assignment to the **Canvas** of ARIN 5305. Appeals regarding the submission of the wrong version after the submission deadline will not be considered.
4. An assignment is an assessment of your individual work. The assignment is subject to the plagiarism policy on the course web page. Plagiarism is a serious offense that can lead to course failure.
5. Submissions after the deadline without pre-approval will not be accepted.
6. You CANNOT make changes to the solution code we provide to you. You can neither change the LLM output nor chat history on the HKUST GenAI OpenWebUI platform.
7. All content should be organized by following the required directory/file structure. You can follow the structure of the example submission content for these tasks in our repository.
8. By the deadline of each stage, please submit the latest zip archive of your repository (incl. all content required by the tasks in each stage) to Canvas. Please also properly commit your progress to your repository every week.
9. You can write tests in either JUnit 4 or 5 formats. Tests generated by Randoop are in JUnit 4 format.

Appendix: Randoop and Eclipse Quick Start

1. Install Randoop and Eclipse

* Install Eclipse IDE 2022-06 or above (for Java Developers) with Java SE 17 (SDK). You can download an Eclipse version (2022-06 or above) at <http://www.eclipse.org/downloads/>. It is bundled with a unit test framework, *JUnit 5*, and a coverage measurement tool, *EclEmma*. (Note that *JUnit 5* plugin supports *JUnit 4*.)

* Follow the documentation of Randoop at <https://randoop.github.io/randoop/> to download the latest version of randoop-all-xxx.jar.

2. Generate Test Cases

After setting up the environment for Randoop, JUnit test case generation can be accomplished in a single command line.

First, you need to go to the directory storing the Java program (Solution.java) and compile this class under test (CUT) into .class files.

Suppose the root folder of the project is $\{\text{ROOT_DIR}\}$, $\{\text{RANDOOP_JAR}\}$ refers to the path of the Randoop .jar file. To compile the Java source code into binary and then use Randoop to generate test cases for our class under test, you can use the following command:

Linux and MacOS (bash):

```
cd {problem_id}
javac Solution.java
java -ea -classpath ${RANDOOP_JAR}:./ randoop.main.Main gentests --testclass=Solution --time-limit=60
```

Windows:

```
cd {problem_id}
javac Solution.java
java -ea -classpath %RANDOOP_JAR%;./ randoop.main.Main gentests --testclass=Solution --time-limit=60
```

Note the randoop parameters used in the command

gentest: Generate test using Randoop

--testclass: The class under test

--timelimit: The time limit for test case generation (in seconds)

--outputlimit: The output test case limit.

You can customize your own options when running Randoop. More options can be found in Randoop's Manual. You can use the given parameters and explore customized parameters to realize task requirements.

3. View and Run the Test Cases

After completing the previous steps, the directory should contain the generated test cases (RegressionTest.java and RegressionTest0.java). Run the generated test cases and measure the coverage

(1) Right-click the main test suite class (RegressionTest0) generated by Randoop (Fig. 1);

(2) Select Coverage as JUnit Test (Fig. 1);

(3) Check coverage and take screenshots (Fig. 2).

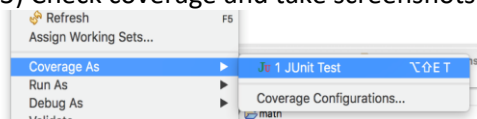


Figure 1 Run JUnit Tests



	Coverage	Covered Branch	Missed Branches	Total Branch
(default package)	100.00%	1/1	0	1
RegressionTest.java	100.00%	1/1	0	1
RegressionTest0.java	100.00%	1/1	0	1
Solution.java	100.00%	1/1	0	1
Solution	100.00%	1/1	0	1

Figure 2 Branch Coverage Measured in Eclipse

4. Further References

- JUnit Cookbook: <https://junit.org/junit5/docs/current/user-guide/>
- EclEmma User Guide: <http://www.eclEmma.org/userdoc/index.html>
- A library of CodeBLEU calculation: <https://github.com/k4black/codebleu>